Industrial Internship Report on

” Banking Information System”

Prepared by

Sumit Tiwari

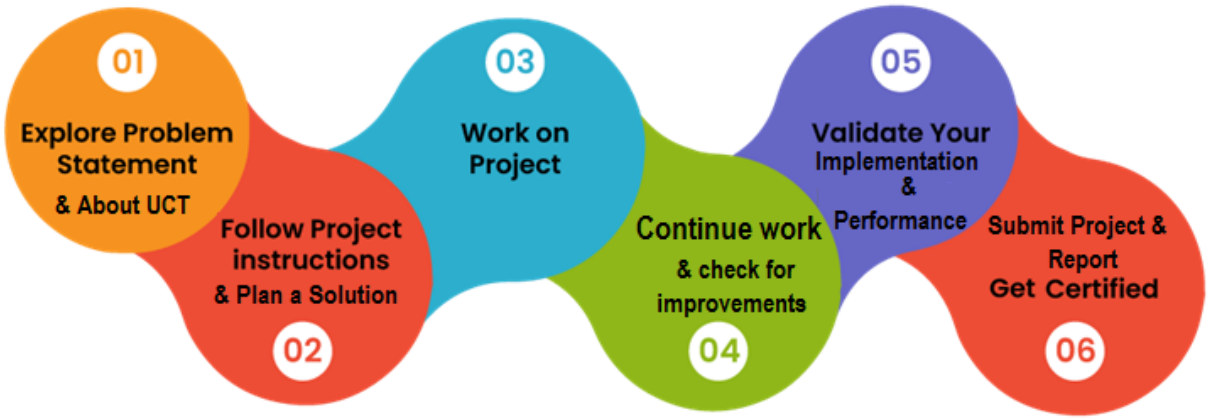
Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time. My project was a Banking Information System — a console-based Core Java application implementing user registration, account management, deposits, withdrawals, fund transfers, account statements, and secure login, built using a layered architecture with custom exception handling and file-based data persistence. This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface

This report summarizes the work completed during my 6-week Core Java Summer Internship under the Summer Internship Program (SIP), conducted in collaboration with upSkill Campus (USC) and UniConverge Technologies Pvt Ltd (UCT). Internships like this are valuable early in a B.Tech program because they bridge the gap between classroom theory and how software is actually built and structured in the industry — something coursework alone rarely covers.

My assigned project was a **Banking Information System**, built as a console-based application in Core Java. The goal was to design a working prototype covering the core operations of a real banking platform: user registration, account management, deposits, withdrawals, fund transfers, account statements, and secure login— all backed by proper error handling and data persistence.



The program was structured across weekly milestones, with progress reports submitted each week covering design decisions, implementation status, and testing. This internship gave me hands-on experience with object-oriented design, exception handling, and file-based persistence in Java — skills that go well beyond what's typically covered in a first or second-year curriculum.

I'd like to thank the USC and UCT team for structuring this program in a way that pushed practical, project-based learning, and for the guidance provided throughou

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

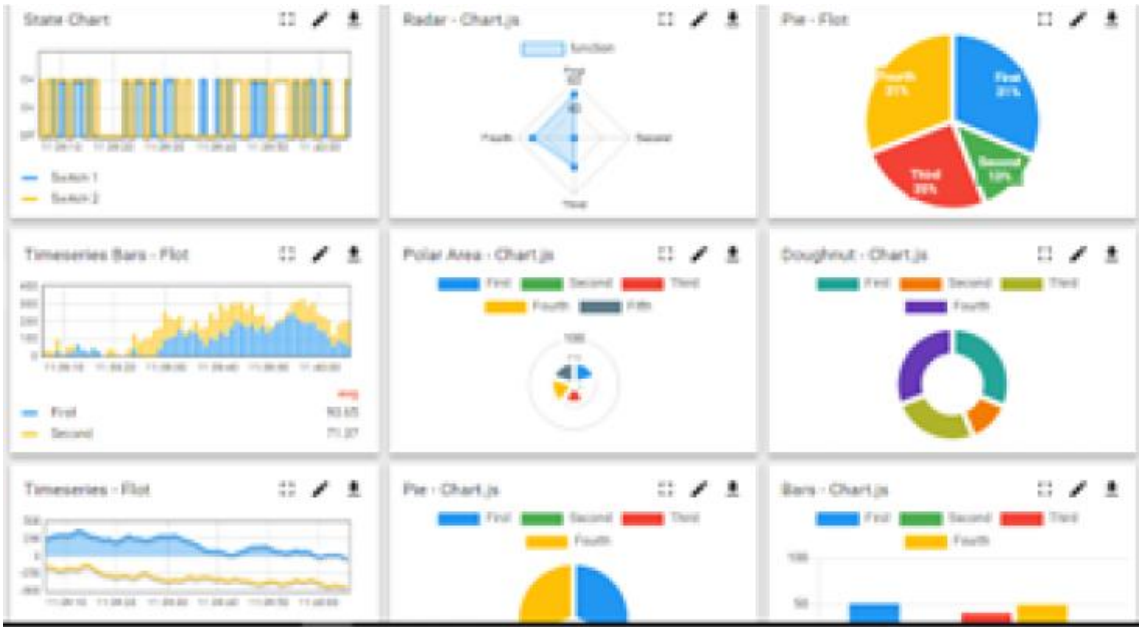
i. UCT IoT Platform (UCT Insight)

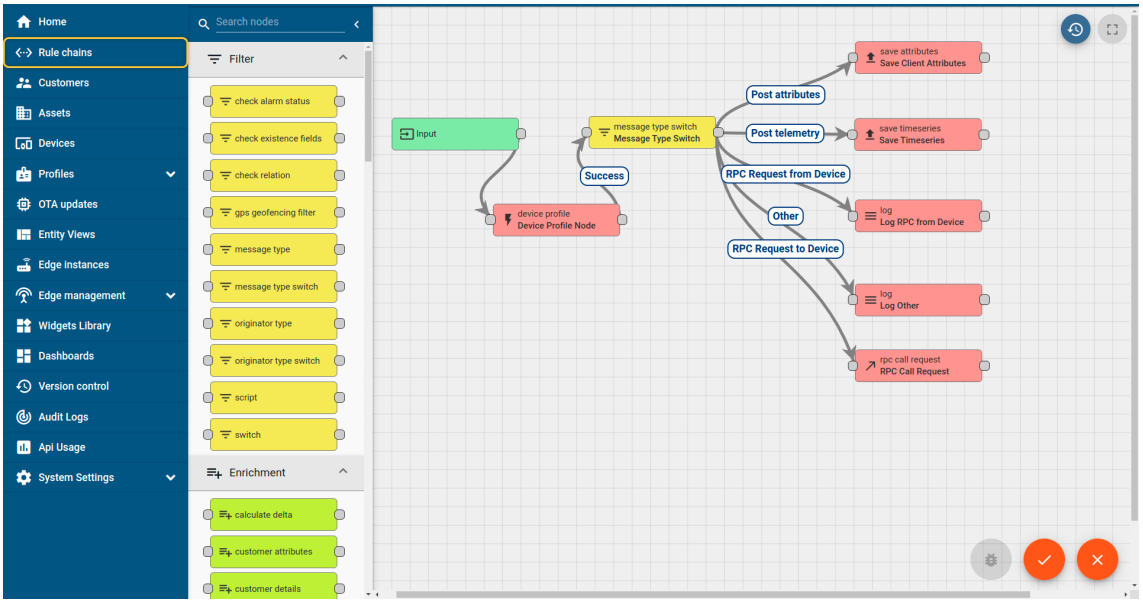
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

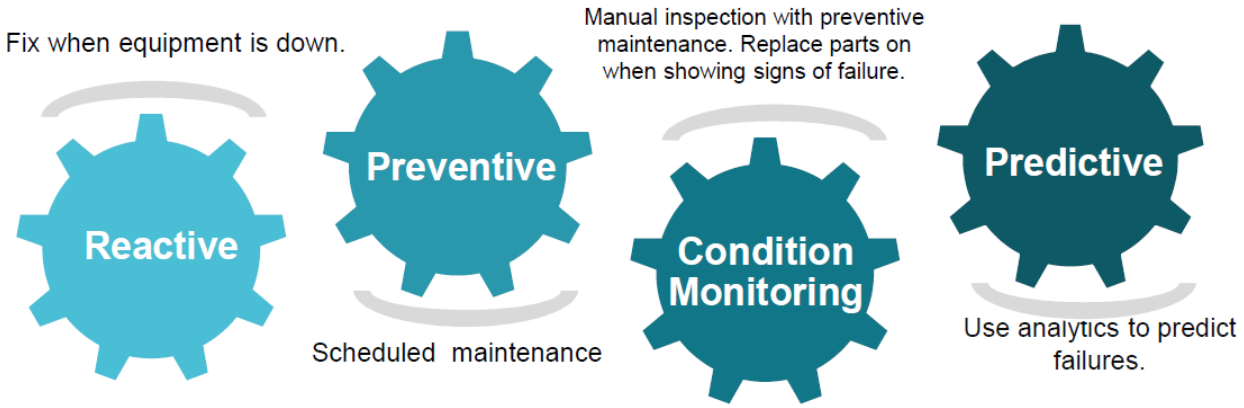
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

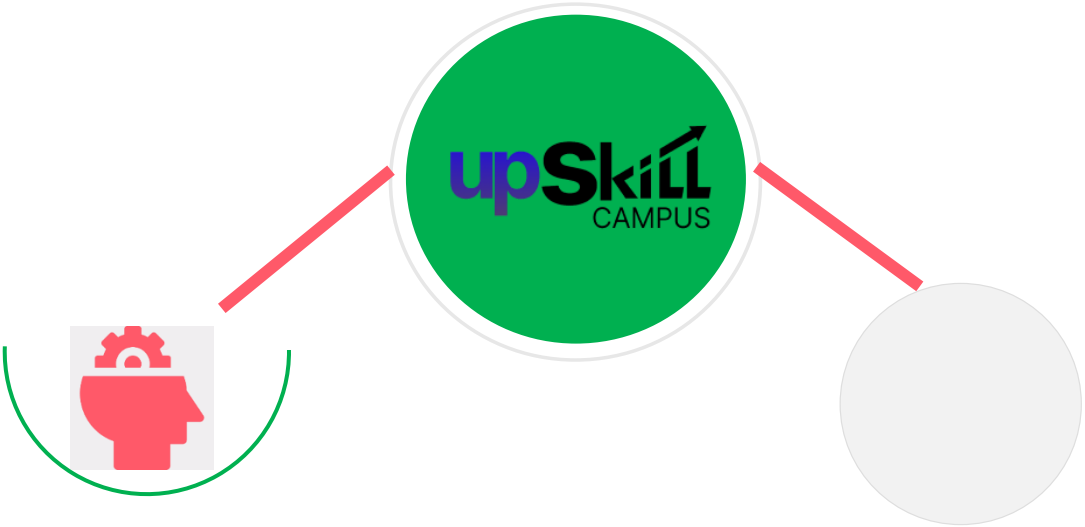
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

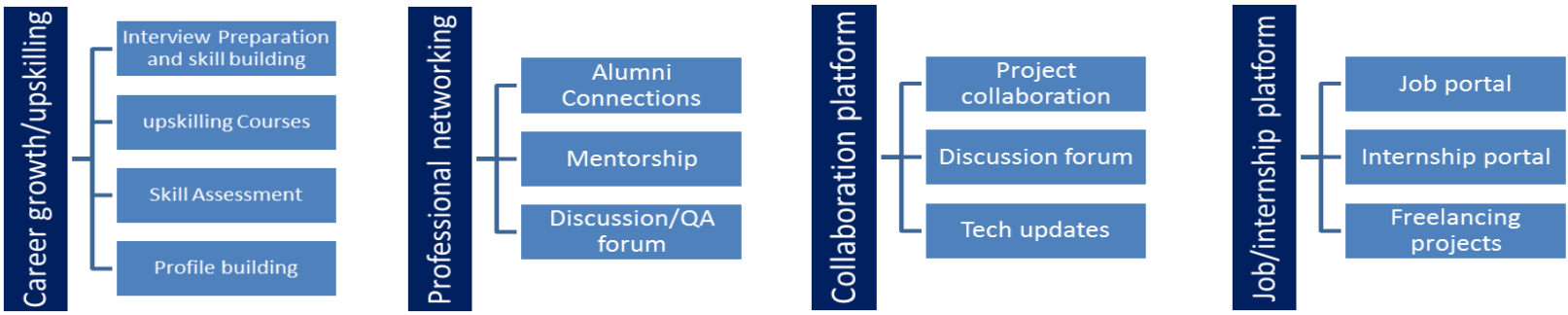
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

3 Problem Statement

Develop a prototype of a Banking Information System in Core Java that provides a working preview of the key functionalities of a real banking system, including user registration, account management, deposits and withdrawals, fund transfers, account statements, password-protected login, and basic error handling — all through a usable interface, with data persisted across sessions.

4 Existing and Proposed solution

Existing solutions: Most real-world banking systems (net banking portals, core banking software used by banks) are large, multi-layered enterprise systems built on databases like Oracle or MySQL, with web/mobile front-ends, strict regulatory compliance, and distributed architecture for scale. These are far beyond the scope of a prototype and not something a beginner project needs to replicate — but the *core operations* (accounts, transactions, balances, statements) are the same at a conceptual level.

Proposed solution: I built a simplified, console-based version that demonstrates these same core operations using a clean, layered Core Java architecture:

- **Model layer** — User, Account, and Transaction classes representing the data
- **Custom exception layer** — dedicated exception classes (InsufficientFundsException, InvalidTransactionException, AccountNotFoundException, AuthenticationException, DuplicateUserException) so error conditions are handled explicitly and predictably rather than through generic error messages
- **Service layer** (BankService) — all business logic (registration, login, deposits, withdrawals, transfers, statements) is centralized here, decoupled from the user interface
- **Persistence layer** — account and transaction data is serialized to a local file, so data survives even after the program is closed and reopened, unlike a purely in-memory prototype

Value addition: Compared to a flat, single-class prototype, this layered design makes the system easier to extend — for instance, the console UI could later be replaced with a GUI or the file-based persistence with a real database, without touching the underlying business logic at all.

4.1 Code submission (Github link)

https://github.com/Sumit-Tiwari008/upskillcampus/blob/main/src/com/bank/BankingInformationSystem.java

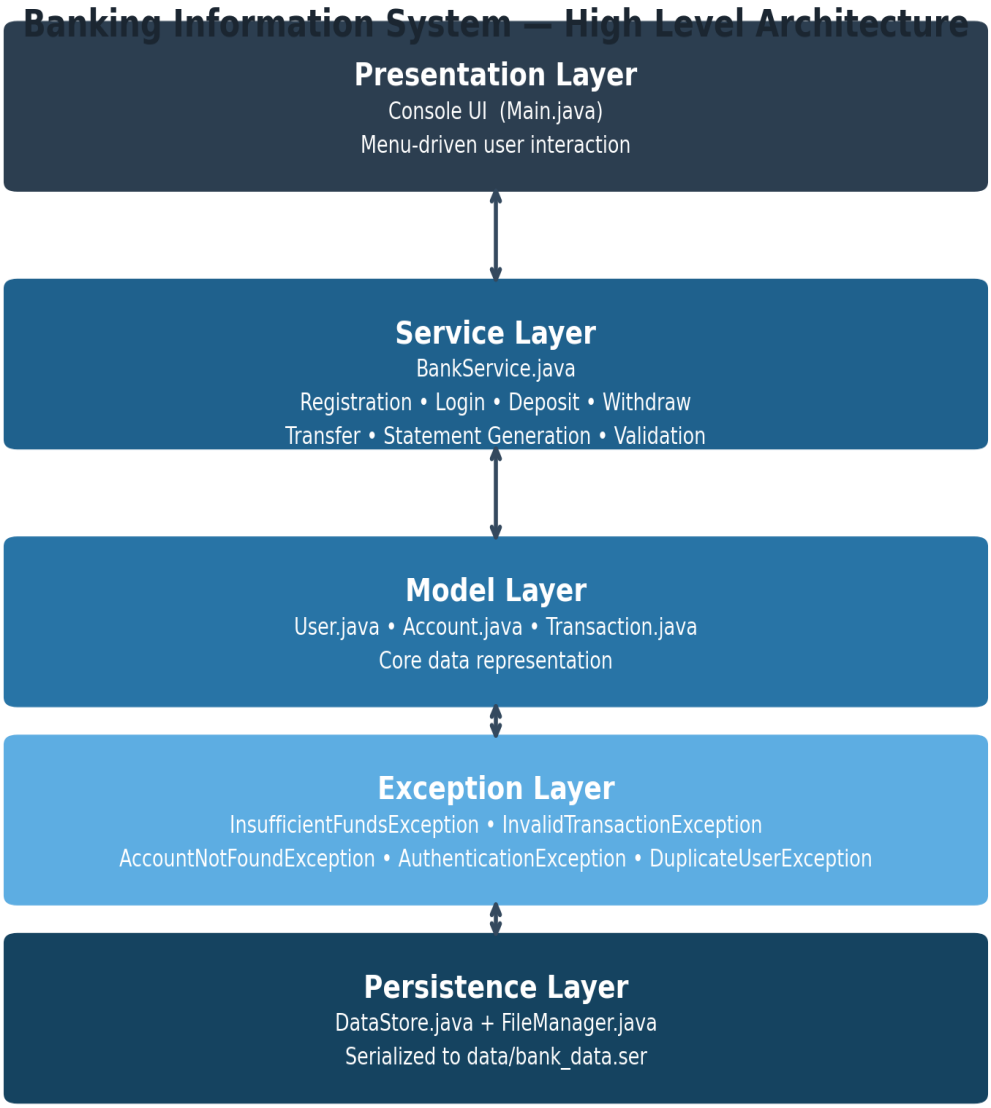
4.2 Report submission (Github link) : .

https://github.com/Sumit-Tiwari008/upskillcampus/blob/main/BankingInformationSystem_Sumit_USC_UCT.pdf

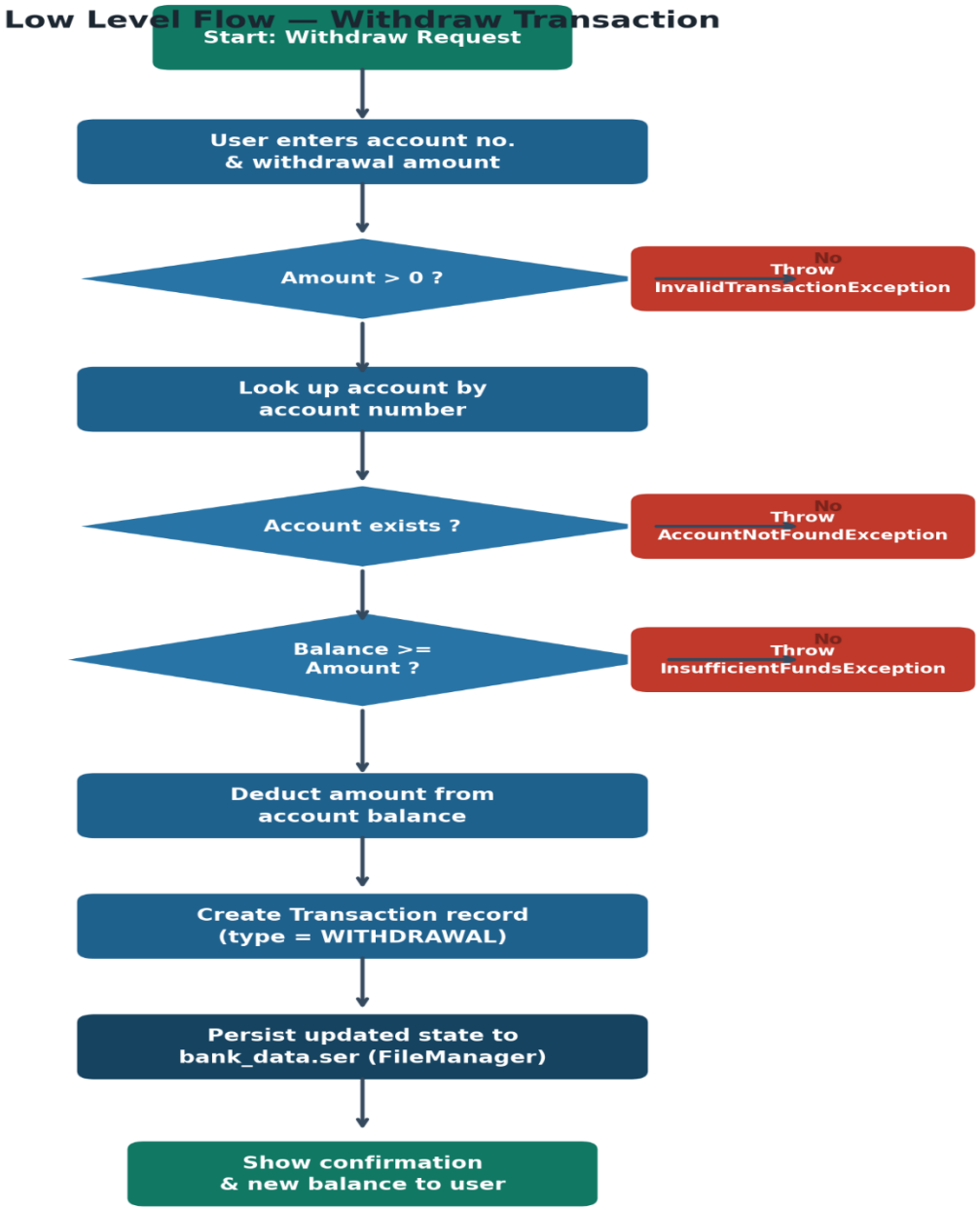
5 Proposed Design/ Model

Given more details about design flow of your solution. This is applicable for all domains. DS/ML Students can cover it after they have their algorithm implementation. There is always a start, intermediate stages and then final outcome.

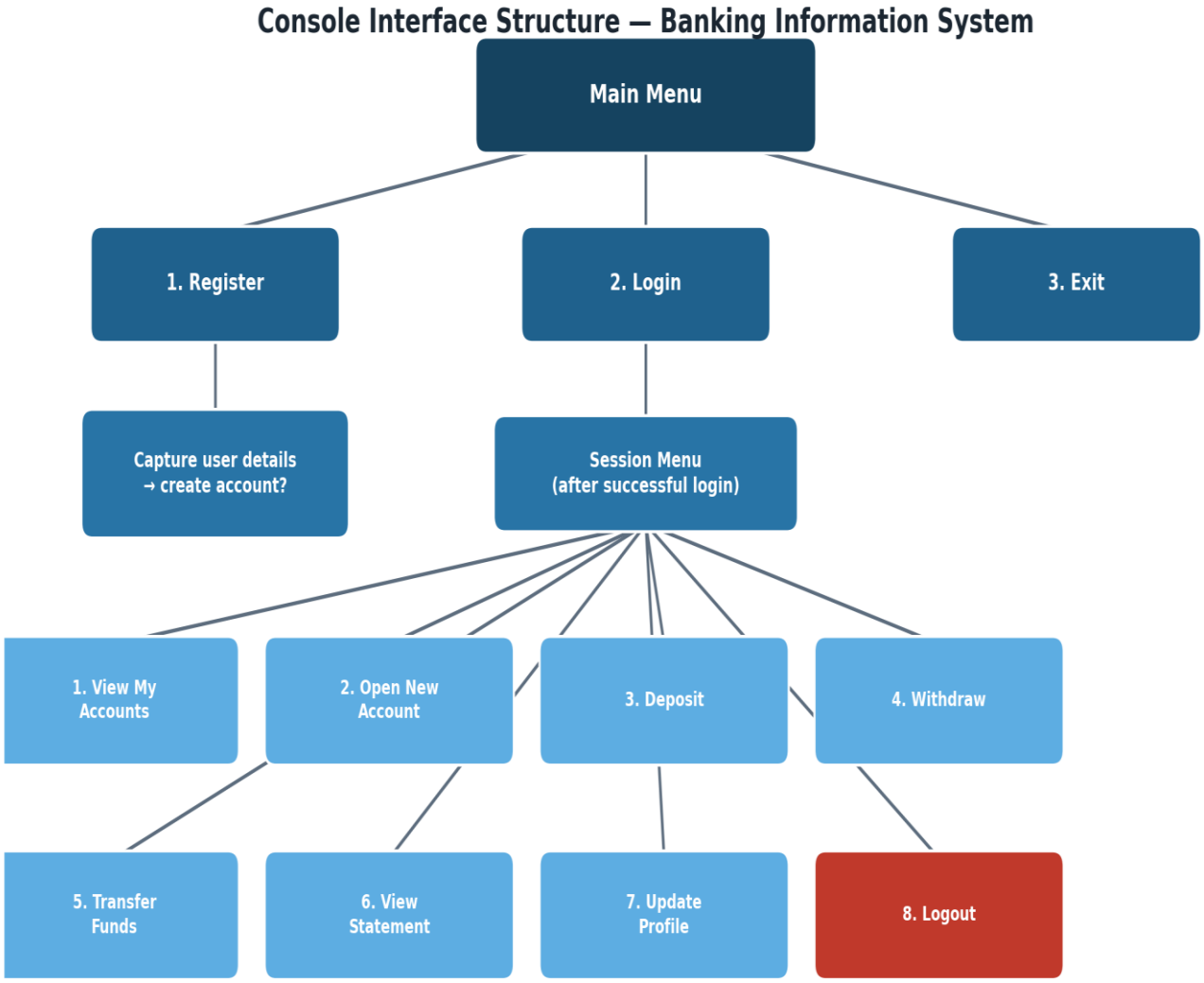
5.1 High Level Diagram (if applicable)



5.2 Low Level Diagram



5.3 Interfaces



6 Performance Test

6.1 Test Plan/ Test Cases

Each test was run by executing the compiled program and manually stepping through the console menu with the relevant inputs, then verifying that the account balance, transaction history, and displayed messages matched the expected outcome.

6.2 Test Procedure

All test cases produced the expected results. Invalid operations (insufficient funds, unknown account numbers, invalid amounts) were correctly caught by the custom exceptions and reported to the user with clear messages rather than crashing the program. Data persisted correctly across multiple runs of the application, confirmed by restarting the program and verifying previously created accounts and balances were still present.

6.3 Performance Outcome

All test cases produced the expected results. Invalid operations (insufficient funds, unknown account numbers, invalid amounts) were correctly caught by the custom exceptions and reported to the user with clear messages rather than crashing the program. Data persisted correctly across multiple runs of the application, confirmed by restarting the program and verifying previously created accounts and balances were still present.

7 My learnings

This project strengthened my understanding of core object-oriented principles — particularly encapsulation and separation of concerns — by requiring me to split logic across model, service, and persistence layers rather than writing everything in one class. I also gained practical experience with Java's exception handling mechanism by designing custom checked exceptions for specific business error conditions, and with file I/O through Java's object serialization for persistence. Beyond the technical skills, this internship taught me to think about a project the way an industry codebase is structured — decoupled, testable, and extensible — rather than just "make it work."

8 Future work scope

Given more time, I would extend this prototype to: replace file-based serialization with a real relational database (MySQL) for more robust and queryable persistence; build a GUI (JavaFX or a web front-end) in place of the console interface; add interest calculation for savings accounts; and add multi-factor authentication or OTP-based login for stronger security, closer to a production banking system.